

Il linguaggio C

Marco Cerbella

Lezione 3 – 28 settembre 2026

1 Linguaggi di programmazione

Definizione. Un *linguaggio di programmazione* è un linguaggio formale che specifica un insieme di istruzioni per un elaboratore. Si usa per realizzare programmi che implementano algoritmi. I linguaggi *ad alto livello* sono vicini al linguaggio umano e lontani dal linguaggio macchina.

I *paradigmi di programmazione* classificano i linguaggi in base alle loro caratteristiche. Il C è *imperativo*: il programmatore indica alla macchina *come* modificare il proprio stato, ad esempio `a = a + 1;`.

2 Dal linguaggio macchina al C

- Il *linguaggio macchina* (sequenze di bit) è l'unico eseguito direttamente dal processore.
- L'*assembly* usa abbreviazioni mnemoniche per le operazioni, ma è legato alla macchina.
- I linguaggi *ad alto livello* sono indipendenti dalla macchina.

Definizione. Un *compilatore* è un programma di sistema che traduce il programma sorgente scritto in un linguaggio ad alto livello in un programma *target* equivalente (assembly/linguaggio macchina). Il programma target viene eseguito in un secondo momento.

In questo corso il compilatore è **gcc**.



3 Le fasi della compilazione con gcc

Quando si lancia gcc, il passaggio da sorgente a eseguibile non è un'unica operazione ma una catena di quattro fasi, ciascuna delle quali può essere fermata con una opzione.

Fase	Comando	Risultato	Cosa fa
Preprocessing	<code>gcc -E main.c -o main.i</code>	<code>main.i</code>	gestisce le righe che iniziano con <code>#</code> : sostituisce i <code>#define</code> , inserisce il contenuto degli <code>#include</code> , toglie i commenti. Il risultato è ancora codice C.
Compilazione	<code>gcc -S main.c</code>	<code>main.s</code>	traduce il C in <i>assembly</i> . Qui emergono gli errori di sintassi e di tipo.
Assemblaggio	<code>gcc -c main.c</code>	<code>main.o</code>	traduce l'assembly in codice macchina (<i>file oggetto</i>). Non è ancora eseguibile.
Linking	<code>gcc main.o -o prog</code>	<code>prog</code>	unisce i file oggetto e le librerie e produce l'eseguibile.

Osservazione. Un file oggetto contiene codice macchina ma le chiamate a funzioni definite in altri file o in libreria (ad esempio `printf`) non hanno ancora un indirizzo. Il *linker* li risolve. Un errore di linking tipico è `undefined reference to 'fun'`: la funzione è stata dichiarata ma non è stata definita in nessuno dei file collegati.

I file oggetto servono per compilare ogni file sorgente separatamente e unirli alla fine. Con due file `main.c` e `altro.c`:

```
gcc -c main.c
gcc -c altro.c
gcc -o prog main.o altro.o
```

oppure in un solo comando `gcc -o prog main.c altro.c`.

4 Caratteristiche del C

Il C è un linguaggio ad alto livello, *procedurale* e di uso generale, progettato (Dennis Ritchie, anni '70) per scrivere sistemi operativi con la massima indipendenza dall'hardware.

Pregi:

- veloce (compilato, vicino alla macchina) e portabile;
- linguaggio piccolo e maturo, con molti strumenti;
- accesso diretto alla memoria e alle funzionalità di basso livello.

Difficoltà:

- si può facilmente fare danni con accessi diretti alla memoria e puntatori;
- la memoria va gestita a mano;
- il codice è più verboso rispetto a linguaggi di scripting come Python.

Osservazione. Il linguaggio C in sé non ha istruzioni di input/output né di gestione della memoria dinamica: queste funzioni sono fornite dalla *libreria standard* (ad esempio `stdio.h`).

5 Standard

Il C è stato standardizzato più volte: C89/C90 (ANSI/ISO), C99, C11, C18. Con gcc si può scegliere lo standard:

```
gcc file.c -std=c11
```

6 Compilati e interpretati

Un linguaggio *compilato* (come C) viene analizzato a fondo dal traduttore prima dell'esecuzione, e quasi tutti i controlli semantici sono fatti a tempo di compilazione. In un linguaggio *interpretato* (come Python) l'interprete resta presente durante l'esecuzione e la maggior parte dei controlli è fatta a tempo di esecuzione.

Osservazione. Un interprete non produce necessariamente codice macchina: spesso esegue direttamente le istruzioni (Python, ad esempio, compila prima in *bytecode* e poi lo interpreta). Essere compilato o interpretato è inoltre una proprietà dell'*implementazione* più che del linguaggio. Un vantaggio del compilato è che molti errori (sintassi, tipi) sono segnalati prima che il programma giri; con l'interprete molti emergono solo quando la riga viene eseguita.